

基于 React 与 Supabase 的地标管理与路线规划系统的设计与实现

Design and Implementation of a Landmark Management and Route Planning System Based on React and Supabase

摘要

随着 Web 地理信息技术与云原生后端服务的快速普及，轻量化、可在浏览器中直接运行的地图应用日益成为出行规划、资产标注与空间分析的重要载体。本文设计并实现了一个基于 B/S 架构的“地标管理与路线规划系统”（Landmark Manager）。系统前端采用 React 19 + TypeScript + Vite 技术栈，以 Leaflet 渲染交互式地图，借助 TailwindCSS 完成响应式布局；后端依托 Supabase 托管的 PostgreSQL 数据库与身份认证服务，并通过行级安全策略（Row Level Security, RLS）实现用户数据隔离；路线规划能力由高德地图 REST API 提供，覆盖步行、驾车与骑行三种出行方式。系统实现了地标的增删改查、基于 Haversine 球面公式的附近搜索与两点测距、Geohash 空间编码、两点路线规划以及含最近邻启发式旅行商（TSP）优化的多站点行程规划，并提供用户注册登录、收藏与历史记录等个性化功能，共划分为地图主页、地标列表、添加地标、路线规划、行程规划、收藏夹与浏览历史七大功能模块。在工程管理上，项目由五名成员组成，采用敏捷迭代（Scrum）与看板（Kanban）相结合的过程模型，配合 Git 版本控制与 Vercel 持续部署。本文系统阐述了需求分析、系统设计（架构模式、数据结构、接口与关键算法）、用例与类建模过程，并以多幅界面截图展示了系统的实际运行效果。实践表明，该系统功能完整、交互流畅、部署便捷，较好地达成了软件工程课程对于需求、设计、实现与文档全流程训练的目标。

关键词：地标管理；路线规划；React；Supabase；Geohash；Haversine 公式；旅行商问题；软件工程

Abstract

With the rapid adoption of web-based geographic information technology and cloud-native backend services, lightweight map applications that run directly in the browser have become an important medium for trip planning, asset annotation and spatial analysis. This paper designs and implements a browser/server (B/S) "Landmark Manager" system for landmark management and route planning. The front end is built with React 19, TypeScript and Vite, renders an interactive map via Leaflet, and uses TailwindCSS for responsive layout. The back end relies on a Supabase-hosted PostgreSQL database and authentication service, isolating user data through Row Level Security (RLS). Route-planning capability is provided

by the AMap REST API, covering walking, driving and cycling modes. The system implements CRUD operations on landmarks, nearby search and point-to-point distance measurement based on the Haversine great-circle formula, Geohash spatial encoding, point-to-point route planning, and multi-stop trip planning with a nearest-neighbor Travelling-Salesman (TSP) heuristic, together with personalized features such as user registration, favorites and history. The application is organized into seven functional modules. In terms of engineering management, the five-member team adopts a hybrid Scrum-and-Kanban agile process supported by Git version control and Vercel continuous deployment. The paper elaborates on requirements analysis, system design (architectural pattern, data structures, interfaces and key algorithms), use-case and class modeling, and demonstrates the running system through a series of interface screenshots. The results show that the system is functionally complete, smoothly interactive and easy to deploy, satisfactorily fulfilling the software-engineering course objectives of full-lifecycle training in requirements, design, implementation and documentation.

Keywords: Landmark management; Route planning; React; Supabase; Geohash; Haversine formula; Travelling Salesman Problem; Software engineering

目 录

一、引言.....	5
1.1 项目背景与意义.....	5
1.2 项目目标.....	5
1.3 团队组成、任务分工与工作量比例.....	6
1.4 报告组织结构.....	6
二、需求分析与开发过程管理.....	7
2.1 功能需求.....	7
2.2 非功能需求.....	7
2.3 开发过程管理模式.....	8
三、系统设计.....	10
3.1 总体架构与结构模式.....	10
3.2 数据结构设计.....	10
3.3 接口设计.....	11
3.4 关键算法.....	12
四、系统建模.....	14
4.1 用例图.....	14
4.2 类图.....	15
五、系统功能演示与界面截图.....	16
5.1 地图主页与 GIS 工具.....	16
5.2 地标列表管理.....	16
5.3 添加地标与 Geohash 实时预览.....	17
5.4 两点路线规划.....	18
5.5 多站点行程规划与 TSP 优化.....	18
六、系统测试与展望.....	20
6.1 功能测试.....	20
6.2 已知局限.....	20
6.3 改进展望.....	21
七、结论.....	21

一、引言

1.1 项目背景与意义

近年来，随着移动互联网与位置服务（Location-Based Service, LBS）的深度融合，地图类应用已经从单纯的“查看地图”演进为集空间数据管理、可视化与路径分析于一体的综合性平台。在城市出行、旅游规划、物流配送以及兴趣点（Point of Interest, POI）标注等场景中，用户不仅需要在地图上浏览地标，更需要对地标进行增删改查、计算空间关系、规划合理的出行路线。与此同时，前端框架（React、Vue）、开源地图库（Leaflet、OpenLayers）以及后端即服务（Backend as a Service, BaaS）平台（如 Supabase、Firebase）的成熟，使得个人开发者与小型团队能够以较低成本快速构建功能完整的 Web 地理信息应用。

本课程项目正是在这一背景下提出的。作为软件工程课程的综合实践，项目要求团队完整经历“需求分析—系统设计—编码实现—测试部署—文档撰写”的软件开发全生命周期，并在真实的协作环境中运用版本控制、迭代管理与持续集成等工程化手段。我们选择实现一个“地标管理与路线规划系统”，既因为它具有清晰的领域模型与丰富的交互场景，便于在有限学期内覆盖软件工程的核心知识点；也因为融合了空间编码（Geohash）、球面距离计算（Haversine）、坐标系转换（WGS-84 ↔ GCS-02）以及旅行商问题（TSP）等具有一定算法深度的内容，能够充分锻炼团队的工程实现与算法应用能力。

从应用价值看，本系统面向有出行规划与地标标注需求的普通用户，提供了一个无需安装、打开浏览器即可使用的轻量化平台；从教学价值看，本项目完整演示了现代 Web 全栈应用的典型技术选型与协作流程，对计算机专业学生理解软件工程方法论具有直接的参考意义。

1.2 项目目标

本项目的总体目标是：基于前后端分离的 B/S 架构，构建一个稳定、易用、可部署的地标管理与路线规划 Web 系统。具体目标可分解为以下几个方面：

1. **功能完整性：**实现地标的增删改查、附近搜索、距离计算、Geohash 编码、两点路线规划与多站点行程规划，并提供用户认证、收藏与历史记录等个性化功能。

2. **算法正确性：**正确实现 Geohash 空间编码、Haversine 球面距离、WGS-84 与 GCJ-02 坐标互转以及最近邻启发式 TSP 路径优化等核心算法。
3. **交互友好性：**提供响应式界面，兼顾桌面端与移动端体验；地图交互流畅，路线与测距结果可视化清晰。
4. **工程规范性：**采用敏捷迭代过程进行团队协作，使用 Git 进行版本控制，借助 Vercel 实现持续部署，保证代码质量与交付效率。

1.3 报告组织结构

本报告其余部分组织如下：第二章分析系统的功能需求与非功能需求，并阐述所采用的软件过程管理模式；第三章给出系统设计，包括总体架构与结构模式、数据结构、接口设计以及关键算法；第四章呈现系统的用例图与类图，建立系统的静态与行为模型；第五章通过界面截图展示系统的功能演示与运行效果；第六章对系统进行测试，并总结项目的不足与展望；第七章为结论。

二、需求分析与开发过程管理

2.1 功能需求

在需求获取阶段，团队通过头脑风暴与场景分析，梳理出系统应当支持的核心业务场景，并据此提炼出功能需求。系统面向“访客”与“注册用户”两类参与者：访客无需登录即可浏览与管理地标、进行空间分析与路线规划；注册用户在此基础上额外获得收藏与历史记录等个性化能力。系统共划分为七个功能模块，各模块的功能需求归纳如表 2-1 所示。

表 2-1 系统功能模块与功能需求

功能模块	核心功能	需求说明
地图主页 Home	交互式地图浏览、附近搜索、距离计算	在 Leaflet 地图上展示全部地标；双击地图设定搜索中心；基于 Haversine 计算半径内地标并排序；选取两地标测算球面距离并在图上绘制测距线。
地标列表 List	表格展示、搜索、筛选、分页、行内操作	以分页表格展示全部地标，支持按名称/分类/Geohash 模糊搜索与分类筛选；每行提供收藏、详情、定位、编辑、删除等操作。
添加地标 Add	表单录入、坐标校验、Geohash 实时预览	录入名称、分类与经纬度；对坐标范围进行校验；随坐标变化实时计算并显示 7 位 Geohash；提交后写入数据库。
路线规划 Route	两点路线规划（步行/驾车/骑行）	选择出行方式与驾车策略；起点支持 GPS、选点或手输；调用高德 API 规划路线，展示距离、耗时与逐步导航，并可跳转外部地图。
行程规划 Trip	多站点行程、TSP 优化	添加至多 5 个途经点并可拖动排序；一键 TSP 最近邻优化访问顺序；逐段规划并以彩色折线展示多段行程。
收藏夹 Favorites	收藏地标/路线/行程的集中管理	登录用户可收藏地标、路线与行程；在收藏夹中分栏展示并支持取消收藏或重新使用。
浏览历史 History	路线/行程历史的自动保存与回放	路线与行程规划成功后自动入库；历史页展示记录详情，支持收藏、加载回放与删除。

2.2 非功能需求

除功能需求外，系统还需满足若干非功能性约束，以保证可用性、性能与可维护

性。结合课程项目的规模与部署环境，团队确定了如表 2-2 所示的非功能需求。

质量属性	具体要求
性能	地图首屏在常规网络下应于 2 秒内完成瓦片加载；列表搜索/筛选/分页通过 useMemo 缓存，交互响应应在毫秒级；路线规划耗时主要取决于第三方 API。
可用性	界面遵循一致的视觉语言，关键操作（删除）提供二次确认；提供数据库连接状态指示与全局 Toast 通知，及时反馈系统状态。
响应式 / 兼容性	采用移动端优先的响应式布局，桌面端侧边栏在移动端转为底部抽屉；支持 Chrome、Firefox、Edge、Safari 等现代浏览器。
安全性	通过 Supabase 行级安全（RLS）策略实现用户私有数据隔离；密钥经环境变量注入而不入库；GPS 功能要求 HTTPS 或 localhost 安全上下文。
可维护性	前后端分离、按模块组织源码、统一的 TypeScript 类型定义；服务层与 UI 层解耦，便于替换地图服务商或数据库。
可移植 / 可部署	前端为纯静态产物，可部署至任意静态服务器或 CDN；通过 Vercel 实现 Git 推送即自动构建与发布。

表 2-2 系统非功能需求

2.3 开发过程管理模式

考虑到课程项目需求相对明确但仍可能在迭代中调整、团队规模较小（五人）且需要频繁沟通的特点，本项目采用以敏捷开发为核心、Scrum 与看板（Kanban）相结合的轻量级过程管理模式，而非瀑布式的一次性交付。这一选择的理由在于：需求可被拆分为相对独立的功能模块（七大模块与若干横切关注点），适合以短周期迭代逐步交付与验证；同时小团队的面对面沟通成本低，能够快速响应变化。

在具体实践中，团队将整个开发周期划分为若干个为期约一周的迭代（Sprint）。每个迭代以一次简短的迭代计划会开始，从产品待办列表（Product Backlog）中选取本迭代要完成的用户故事，形成迭代待办列表（Sprint Backlog）；迭代过程中通过简短的每日站会同步进度与障碍；迭代结束时进行评审与回顾，演示已完成功能并总结改进点。各迭代的大致主题如表 2-3 所示。

迭代	主题	主要交付物
Sprint 1	脚手架与数据底座	搭建 Vite + React + TS 工程与 Tailwind；建立 Supabase landmarks 表与 RLS；完成地图主页与地标加载。
Sprint 2	地标 CRUD 与列表	添加/编辑/删除地标、Geohash 实时预览；列

		表搜索、筛选、分页与行内操作。
Sprint 3	空间分析与路线规划	Haversine 附近搜索与测距；amap 服务层与坐标转换；两点路线规划（步行/驾车/骑行）。
Sprint 4	行程规划与个性化	TSP 多站点行程；用户注册登录与 AuthContext；收藏、历史记录与自动保存。
Sprint 5	联调、测试与部署	响应式适配、功能测试清单执行、缺陷修复；Vercel 持续部署与文档撰写。

表 2-3 迭代计划与主要交付物

在配置管理方面，团队以 Git 作为版本控制工具，采用基于功能分支（feature branch）的协作模型：成员在各自分支上开发，完成后通过合并请求（Pull Request）并经至少一名其他成员评审后合入主干。持续部署由 Vercel 承担，主干每次更新即触发自动构建与上线，使团队能够随时获得可访问的最新版本，显著缩短了反馈回路。看板则用于可视化任务状态（待办、进行中、评审、完成），帮助团队识别瓶颈、控制在制品数量。

三、系统设计

3.1 总体架构与结构模式

系统整体采用前后端分离的 B/S（Browser/Server）架构。与传统三层架构不同的是，本系统在持久层与业务层之间引入了一层 Redis GEO 空间索引缓存层，用于加速"按经纬度+半径搜索邻近地标"这一核心 GIS 查询。前端内部遵循"UI 组件层—逻辑/服务层—数据模型层"的分层组织方式，是一种关注点分离（Separation of Concerns）的体现：表示层只负责渲染与交互，业务逻辑集中于服务层与全局状态管理，数据访问则封装在数据库客户端与服务封装中。系统的总体架构如图 3-1 所示。

在前端，React 19 单页应用（SPA）由 Vite 构建，TailwindCSS 提供原子化样式，Leaflet 与 react-leaflet 协同渲染交互式地图。应用通过 Supabase JS SDK 与云端的 PostgreSQL 数据库及认证服务通信，通过 HTTP 直接调用高德地图 REST API 完成路线规划。Vercel Serverless Functions 作为中间 API 层，承担 Redis GEO 查询、索引同步与健康监控职责，使前端与 Redis 之间的交互保持在服务端内部，避免将 Redis 连接暴露于浏览器，部署交由 Vercel 自动完成。

层次	主要构成	职责
表示层（UI）	Header、InteractiveMap、各页面组件	渲染界面、处理用户交互、通过 props 与回调与上层通信。
状态/逻辑层	App.tsx（提升状态）、AuthContext	集中管理全局状态与 Tab 路由，编排 CRUD 与认证流程。
服务层	services/amap.ts、utils.ts	封装路线规划、坐标转换、TSP、Geohash 与 Haversine 等纯逻辑。
数据访问层	lib/supabase.ts	初始化 Supabase 客户端，统一数据库与认证调用入口。
API 中间层	Vercel Serverless Functions（api/目录）+ vite-api-plugin.ts	代理 Redis GEO 查询、索引同步与健康监控；屏蔽 Redis 连接细节
缓存/空间索引层	Redis（ioredis）GEO Sorted Set	以 52-bit Geohash 整数为 score 的空间索引，支持 GEOSEARCH 半径查询
持久化/外部服务	Supabase PostgreSQL、Auth、高德 API	数据持久化、用户认证、行级安全与第三方路线服务。

表 3-1 系统分层结构与职责

在状态管理模式上，项目没有引入 Redux 等重量级状态库，而是采用 React 内置的"状态提升（Lifting State Up）+ Context"方案。全局业务状态集中在 App.tsx 的

AppInner 组件中，经由 props 向下传递，事件回调向上汇聚；唯有跨层级、强相关的用户认证状态通过 AuthContext 以 Provider/Consumer 模式注入整棵组件树。配合 useCallback 与 useMemo 进行引用稳定与计算缓存，既保持了数据流的清晰单向性，又避免了不必要的重渲染，契合中小型应用的复杂度。

在认证守卫设计上，AuthContext 提供了 requireAuth(action) 模式：调用者传入一个需登录后执行的回调函数，若用户已登录则立即执行，否则先打开登录弹窗并将回调挂起，待登录成功后再自动执行。这种“延迟授权”的设计解耦了业务操作与认证流程，避免了在每个需要登录的功能中重复编写“检查登录→弹窗→重试”的逻辑。

在 API 层的架构设计上，系统采用了开发/生产双环境路由策略：开发环境下，vite-api-plugin.ts 作为 Vite 插件直接拦截 /api/* 请求，在 dev server 进程内动态 import handler 模块并处理请求，无需额外启动 API 服务器；生产环境下，vercel.json 中的 rewrites 规则将 /api/* 路由到 Vercel Serverless Functions 对应的 handler 文件，实现零配置部署切换。

3.2 数据结构设计

系统的持久化数据存储于 Supabase 托管的 PostgreSQL 数据库，共设计四张表：landmarks（地标主表）、favorites（收藏）、saved_routes（保存的路线）与 saved_trips（保存的行程）。其中 landmarks 为全局共享数据，对所有访客开放读写；其余三张表均为用户私有数据，通过行级安全策略约束 auth.uid() = user_id，确保用户只能访问自己的记录。核心表 landmarks 的结构如表 3-2 所示。

字段	类型	说明
id	UUID（主键，gen_random_uuid）	地标唯一标识，自动生成。
name	TEXT NOT NULL	地标名称。
category	TEXT NOT NULL	地标分类，如“Global Landmark”。
latitude	DOUBLE PRECISION	纬度，取值 -90 ~ 90。
longitude	DOUBLE PRECISION	经度，取值 -180 ~ 180。
geohash	TEXT NOT NULL	7 位 Geohash 空间编码。
created_at	TIMESTAMPTZ DEFAULT NOW()	创建时间，自动写入。

表 3-2 landmarks 表结构

用户私有的三张表则在主键、外键之外携带各自的业务字段：`saved_routes` 记录起终点坐标、出行方式、驾车策略与序列化的完整路线数据（`route_data`，JSONB）；`saved_trips` 记录有序的途经点 ID/名称列表、可选的 GPS 起点与各段路线数据（`segments_data`，JSONB）；`favorites` 通过 `UNIQUE(user_id, landmark_id)` 约束防止同一用户重复收藏。为加速按用户与按地标的查询，系统在 `user_id` 与 `landmark_id` 等列上建立了 B-Tree 索引（如 `idx_favorites_user_id`、`idx_saved_routes_user_id` 等）。

在前端，上述数据模型以 TypeScript 接口的形式集中定义于 `types.ts`，与数据库表一一对应，保证了跨层的类型一致性。例如地标的核心接口定义如下：

```
export interface Landmark {
  id: string; name: string; category: string;
  latitude: number; longitude: number;
  geohash: string; created_at?: string;
}
```

此外，路线规划相关的运行期数据结构（如 `RouteResult`、`RoutePath`、`RouteStep`）则定义于服务层 `amap.ts`，以统一封装高德 API 返回的多条备选路线、逐步导航指令与折线坐标，屏蔽不同出行方式 API 在响应格式上的差异。

属性	说明
Key	landmarks:geo
类型	GEO 空间索引（底层为 Sorted Set）
Member	地标 UUID（字符串）
Score（内部）	52 - bit Geohash 整数编码
命令示例	GEOADD landmarks:geo -73.9857 40.7484 "uuid - empire - state"
查询示例	GEOSEARCH landmarks:geo FROMLONLAT -74.0 40.7 BYRADIUS 10 km ASC COUNT 100 WITHDIST
同步策略	全量：Supabase → Pipeline GEOADD；增量：前端 CRUD 后 fire - and - forget 调用 /api/geo - update

表 3.2'：Redis GEO 空间索引数据结构

Redis GEO 索引与 PostgreSQL landmarks 表构成"主存储 + 空间加速索引"的双层数

据模型：PostgreSQL 为权威数据源（Source of Truth），Redis 为只读查询加速层，二者通过全量同步与增量更新保持最终一致性。

3.3 接口设计

系统的接口可分为内部服务接口、外部 API 接口与 Redis 运维接口三类。内部服务接口指服务层向 UI 层暴露的函数式接口，是 UI 与业务逻辑之间的契约；外部 API 接口则指系统与 Supabase 及高德开放平台之间的通信约定；Redis 运维接口则是维护 Redis GEO 索引与 PostgreSQL 之间数据一致性的管理 API。

3.3.1 内部服务接口

在内部接口方面，服务层对外暴露的核心函数及其职责如表 3-3 所示。这些接口以纯函数或 Promise 的形式提供，输入输出均以 TypeScript 类型严格约束，UI 层只需关心“调用什么、得到什么”，无需了解内部实现细节。

接口签名	所属模块	说明
<code>planRoute(opts: RoutePlanOptions): Promise</code>	<code>amap.ts</code>	统一路线规划入口，按出行方式（walking/driving/bicycling）分派到对应实现，自动完成坐标转换与响应解析
<code>optimizeWaypointOrder(startLng, startLat, points: [number,number][]): number[]</code>	<code>amap.ts</code>	最近邻 TSP 启发式，返回优化后的途经点访问顺序索引数组
<code>wgs84ToGcj02(lng, lat): [number, number]</code>	<code>amap.ts</code>	WGS - 84 → GCJ - 02 坐标系转换（发往高德 API 前调用）
<code>gcj02ToWgs84(lng, lat): [number, number]</code>	<code>amap.ts</code>	GCJ - 02 → WGS - 84 坐标系转换（解析高德返回折线时调用）
<code>encodeGeohash(lat, lng, precision=7): string</code>	<code>utils.ts</code>	将经纬度编码为 BASE32 Geohash 字符串
<code>calculateHaversineDistance(lat1, lon1, lat2, lon2): number</code>	<code>utils.ts</code>	计算两点间球面大圆距离（km），结果

		保留三位小数
<code>formatDuration(seconds): string</code>	<code>amap.ts</code>	秒数格式化为可读的"X hr Y min"格式
<code>formatDistance(meters): string</code>	<code>amap.ts</code>	米数格式化为"X km"或"X m"格式
<code>buildExternalMapUrl(...): {amap, bmap, apple}</code>	<code>amap.ts</code>	构建高德、百度、Apple Maps 三方的外部导航链接
<code>requireAuth(action: () => void): void</code>	<code>AuthContext</code>	权限守卫：已登录则执行回调，否则唤起登录弹窗并挂起回调

表 3-3 服务层核心接口

3.3.2 外部 API 接口

系统通过 Supabase JS SDK 以声明式方式访问数据库，所有 CRUD 操作均通过 `supabase.from('表名').方法('*')` 完成：

操作类型	SDK 调用示例	说明
查询全部地标	<code>supabase.from('landmarks').select('*').order('created_at', ascending: false)</code>	按创建时间降序
新增地标	<code>supabase.from('landmarks').insert([item]).select()</code>	插入后返回新记录
更新地标	<code>supabase.from('landmarks').update(updates).eq('id', id).select()</code>	按 ID 精确更新
删除地标	<code>supabase.from('landmarks').delete().eq('id', id)</code>	按 ID 精确删除
用户认证	<code>supabase.auth.signUp({email, password}) / supabase.auth.signInWithPassword({email, password})</code>	邮箱注册/登录
用户登出	<code>supabase.auth.signOut()</code>	清除会话
收藏操作	<code>supabase.from('favorites').insert/delete</code>	基于 <code>user_id</code> + <code>landmark_id</code>
路线历史	<code>supabase.from('saved_routes').select/insert/update/delete</code>	基于 <code>user_id</code>

操作类型	SDK 调用示例	说明
行程历史	<code>supabase.from('saved_trips').select/insert/update/delete</code>	基于 <code>user_id</code>

3.4 Redis 空间索引层设计与实现

3.4.1 连接管理

关键设计决策：

单例模式：模块级 `let redis` 变量保证整个 Serverless Function 实例生命周期内复用同一 TCP 连接。在 Vercel 的 Serverless 环境中，冷启动时创建连接，后续同实例的请求直接复用，避免了频繁握手的开销。

惰性连接（lazyConnect）：设置 `lazyConnect: true` 使得构造函数不立即建立 TCP 连接，而是延迟到首次命令执行时。这在 Vercel Serverless 冷启动场景中尤为关键——如果 Redis 恰好不可用，构造函数不会抛出异常阻塞模块加载，错误将在实际调用时以更可控的方式暴露。

TLS 自适应：通过检测 `REDIS_URL` 是否以 `rediss://` 开头自动启用 TLS 加密，兼容 Upstash、Redis Cloud 等托管服务。`rejectUnauthorized: false` 用于自签名证书环境。

错误静默：`.on('error', () => {})` 防止 `ioredis` 默认的"未捕获 `error` 事件导致进程崩溃"行为。在 Vercel 环境下，这确保 Redis 临时不可用不会导致 Function 实例崩溃，系统可通过降级策略继续提供服务。

3.4.2 全量同步机制

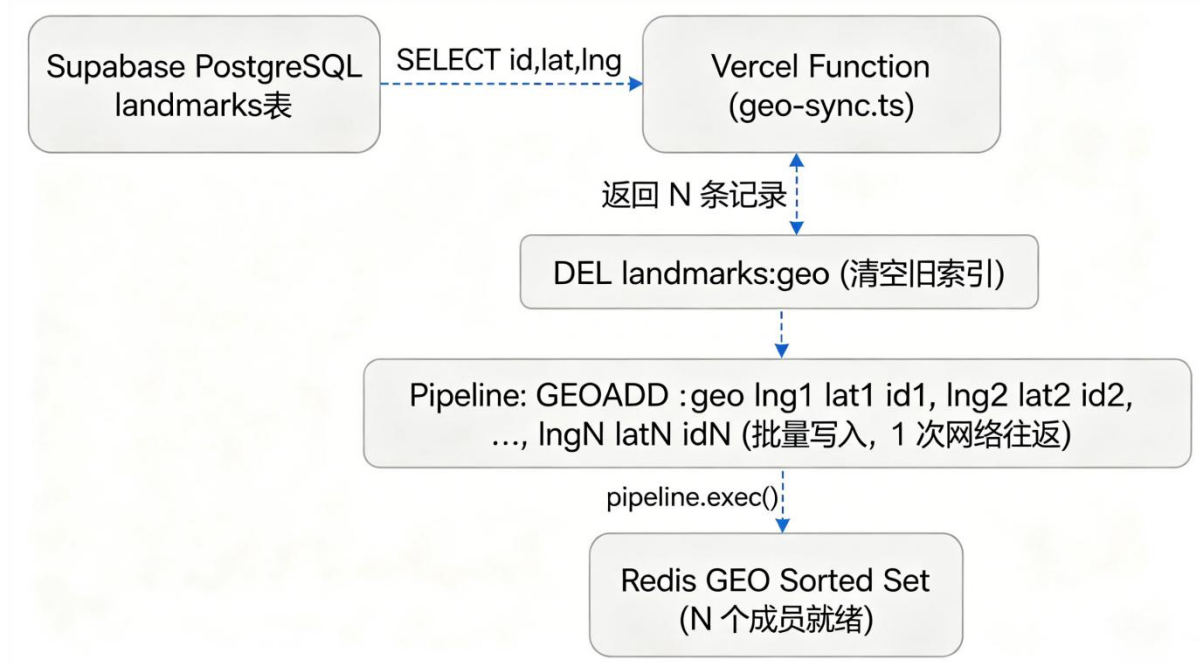


图 3-2: Redis 全量同步流程

Pipeline 的使用是性能瓶颈的关键优化：每个 GEOADD 单独发送需要 N 次网络往返（RTT），使用 Pipeline 将 N 个命令打包为一个 TCP 报文发送，仅需 1 次 RTT。在 N=1000 的场景下，以典型的 50ms RTT 计算，Pipeline 将同步耗时从 ~50 秒压缩到 ~50 毫秒。

3.4.3 增量更新机制

与全量同步互补，增量更新用于维持 Redis 索引的实时性。设计要点：fire-and-forget 模式下不 await 返回值，不处理错误（.catch(() => {}）），确保 Redis 的不可用不会影响用户的主流程体验。即使 Redis 宕机，地标的增删改仍然成功，仅邻近搜索功能暂时降级。Redis 索引可能短暂落后于 PostgreSQL（毫秒级延迟），但下一次全量同步或手动触发同步后可完全修复。

3.4.4 就近搜索与优雅降级

前端 handleSearchNearby 函数实现了双层回退策略，确保了系统的鲁棒性：Redis 正常工作时享受高性能空间索引；Redis 不可用时系统仍可通过纯前端计算完成邻近搜索，用户体验不中断，仅搜索结果中会标注 "local" 以示区别。

3.4.5 健康监控

Header 组件通过 React useEffect 在初始化时调用 /api/geo-test，并在界面右上角展示 Redis 状态指示灯：绿色圆点 + "Redis GEO: N entries"代表 Redis 连接正常；红色圆

点 + "Redis GEO: offline"代表 Redis 不可用

这种可视化的健康监控使开发者和运维人员能一眼判断 Redis 层是否正常工作。

3.4 关键算法

系统在空间编码、距离计算、坐标转换与路径优化四个方面应用了具有一定深度的算法，下面分别说明其原理与实现要点。

3.4.1 Geohash 空间编码

Geohash 是一种将二维经纬度坐标编码为一维 Base32 字符串的空间索引算法。其核心思想是对经度区间 $[-180, 180]$ 与纬度区间 $[-90, 90]$ 交替进行二分：偶数位处理经度、奇数位处理纬度，每一步根据坐标落在区间中点的哪一侧输出比特 1 或 0，并相应收缩区间；每累积 5 个比特即查 Base32 字符表（0-9、b-z 去除 a/i/l/o）编码为一个字符，直至达到目标精度。本系统采用 7 位精度，对应约 ± 76 米的定位误差。该编码具有“前缀相似即空间相邻”的良好性质，可用于快速的区域查询；在添加与编辑地标时，系统会随坐标输入实时计算并预览 Geohash，给予用户直观反馈。

3.4.2 Haversine 球面距离

为准确计算地表两点间的距离，系统采用 Haversine 公式而非平面欧氏距离，以考虑地球曲率。设两点纬度为 ϕ_1 、 ϕ_2 （弧度），经度差为 $\Delta \lambda$ 、纬度差为 $\Delta \phi$ ，地球平均半径 $R = 6371$ km，则距离 d 由下式给出：

$$a = \sin^2(\Delta \phi / 2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta \lambda / 2)$$
$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}), \quad d = R \cdot c$$

该算法被应用于 GIS 工具栏的两点测距、附近搜索中各地标到搜索中心的距离排序，以及 TSP 路径优化中点间距离的度量。实现中结果保留三位小数，兼顾精度与可读性。

3.4.3 WGS-84 与 GCJ-02 坐标转换

中国境内的电子地图普遍采用 GCJ-02（国测局坐标系，俗称“火星坐标系”），它在 WGS-84（GPS 全球标准）基础上施加了非线性偏移加密。由于高德 API 使用 GCJ-02 而 Leaflet 使用 WGS-84，系统必须在二者间互转。转换时首先判断坐标是否位于中国境外（境外直接返回原值），随后基于一组以三角函数与多项式构成的偏移函数计算纬度偏移 $dLat$ 与经度偏移 $dLng$ ，并结合椭球长半轴 $A = 6378245$ 、第一偏心率平方 $EE \approx 0.006694$ 将其折算为实际角度偏移；WGS→GCJ 加偏移，GCJ→WGS 减偏

移。该转换在发送路线请求与解析返回折线时被反复调用。

3.4.4 最近邻启发式旅行商（TSP）路径优化

多站点行程规划本质上是旅行商问题（TSP）：在给定若干途经点的情况下，寻找访问全部点一次的最短路径。精确求解的时间复杂度为 $O(n!)$ ，难以在线实时计算，故系统采用最近邻贪心启发式：自起点出发，每一步在尚未访问的点中选择距当前点最近者并移动过去，重复直至全部访问完毕。其时间复杂度为 $O(n^2)$ 、空间复杂度 $O(n)$ 。由于系统将途经点限制为至多 5 个， n 较小，该近似解在绝大多数情形下都能给出令人满意的访问顺序，兼顾了求解质量与实时性。算法核心实现如下：

```
for (let i = 0; i < n; i++) {  
  let bestIdx = -1, bestDist = Infinity;  
  for (let j = 0; j < n; j++) {  
    if (visited.has(j)) continue;  
    const d = haversine(curLat, curLng, pts[j][1], pts[j][0]);  
    if (d < bestDist) { bestDist = d; bestIdx = j; }  
  }  
  visited.add(bestIdx); order.push(bestIdx); /* 移动到 bestIdx */  
}
```

3.5.5 Redis GEOSEARCH 空间半径搜索

Redis GEO 类型的底层实现是将经纬度坐标编码为 52-bit Geohash 整数，以该整数作为 Sorted Set 的 score 存储成员。GEOSEARCH 命令在此基础上实现了空间半径搜索：

```
GEOSEARCH landmarks:geo  
FROMLONLAT <lng> <lat>  — 以给定经纬度为中心点  
BYRADIUS <radius> km    — 搜索半径（km）  
ASC                      — 按距离升序排列  
COUNT 100              — 最多返回 100 条  
WITHDIST                 — 附带每条的精确距离
```

Redis 内部通过 Geohash 整数编码将二维空间搜索转化为一维区间查询。首先计算中心点的 52-bit Geohash，然后根据半径推算出需要查询的 Geohash 前缀范围，在 Sorted Set 中执行 ZRANGEBYSCORE 扫描该范围内的成员，最后对候选集计算精确的 Haversine 距离并过滤、排序。整个过程的时间复杂度为 $O(\log N + M)$ ，相比前端全量遍历的 $O(N)$ 有显著优势。

与系统自研的 Haversine 函数相比，Redis GEOSEARCH 的优势在于：（1）利用 Sorted Set 的跳表索引将查询降至对数级；（2）所有计算在服务端内存中完成，避免了将全部地标数据传输到前端的网络开销；（3）WITHDIST 选项返回的已是最精确的大圆距离（Redis 内部使用 Haversine 或更精确的 Vincenty 公式）。

四、系统建模

为在编码之前厘清系统的功能边界与静态结构，团队采用统一建模语言（UML）对系统进行用例建模与类建模。用例图刻画系统对外提供的功能及参与者与功能之间的关系，是需求的可视化表达；类图则描述系统内部的关键抽象及其关联、依赖与组合关系，是设计的静态视图。

4.1 用例图

系统的参与者包括访客（Guest）、注册用户（Member）以及作为外部系统的高德地图 API 与 Supabase 云数据库。访客可执行浏览地图、查询筛选列表、增删改查地标、附近搜索、距离计算、两点路线规划、多站点行程规划以及注册登录等用例；注册用户在继承访客全部能力的基础上，扩展出收藏、管理浏览历史与跳转外部导航等用例。其中，“添加地标”“编辑地标”均包含（«include»）“Geohash 编码预览”用例；“多站点行程规划”可扩展（«extend»）出“TSP 路径优化”，“两点路线规划”可扩展出“跳转外部地图导航”。系统用例图如图 4-1 所示。

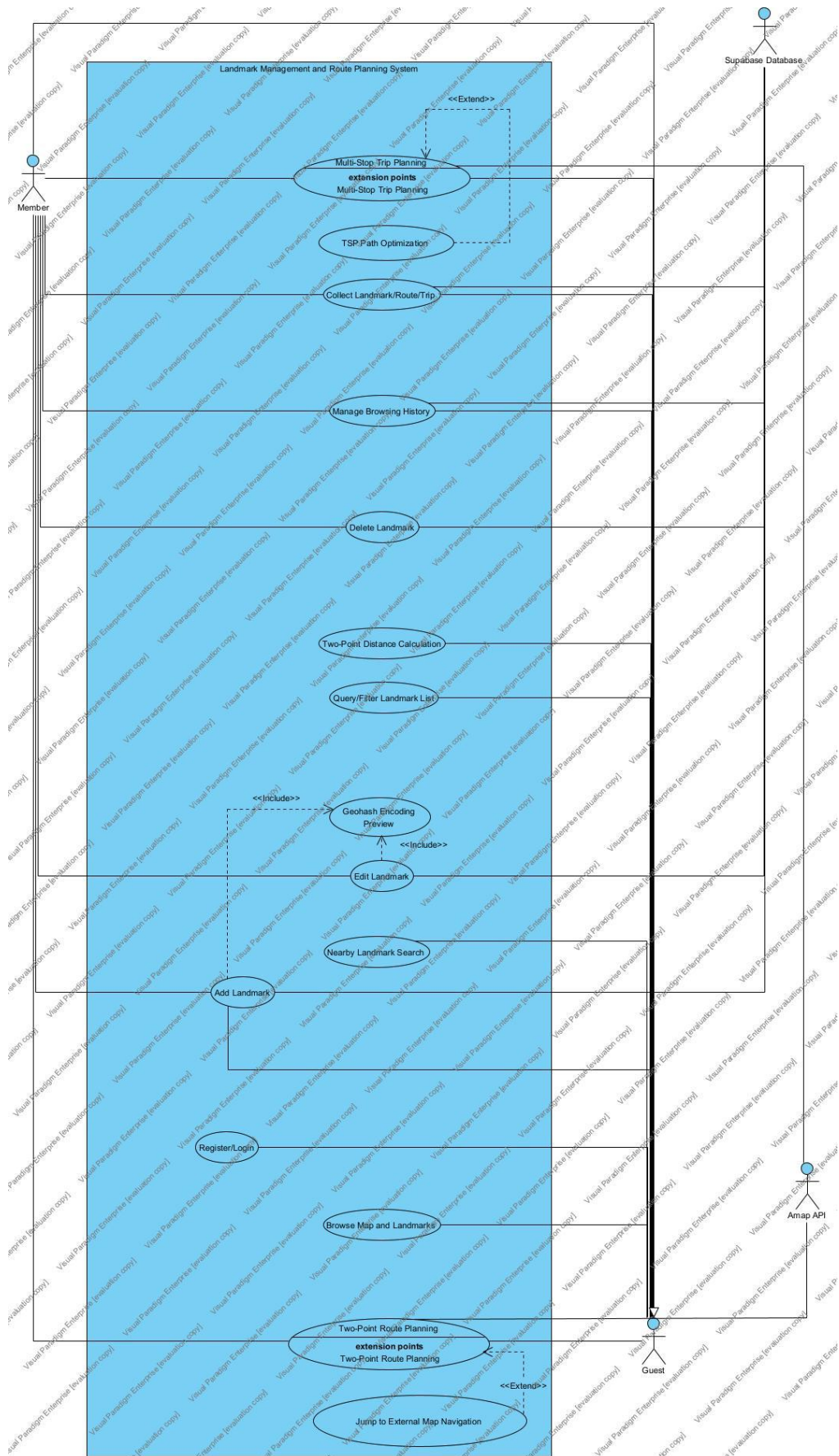


图 4-1 地标管理系统用例图

Figure 4-1 Use Case Diagram of the Landmark Manager System

4.2 类图

系统的类图（图 4-2）从三个层次刻画了核心抽象。最上方是与数据库表对应的数据模型接口（Landmark、Favorite、SavedRoute、SavedTrip）以及路线规划的运行期类型 RouteResult；中间是逻辑与服务层，包括封装路线与坐标逻辑的 AMapService 模块、提供 Geohash 与 Haversine 的 Utils 模块、管理认证状态的 AuthContext 以及统一数据访问的 supabase 客户端；最下方是 UI 组件层，以 App（AppInner）为核心容器，组合 InteractiveMap、Route/TripPlanner、List/Add/Edit 等子组件。

图中以实心菱形表示组合关系（App 拥有并管理各子组件的生命周期），以带箭头实线表示关联（如 App 管理 Landmark 集合、AMapService 返回 RouteResult），以虚线箭头表示依赖（如 Planner 依赖 AMapService.planRoute、组件依赖 Utils 的算法、AuthContext 依赖 supabase 完成认证）。该结构清晰地体现了 UI、逻辑与数据三层之间单向、低耦合的依赖方向。

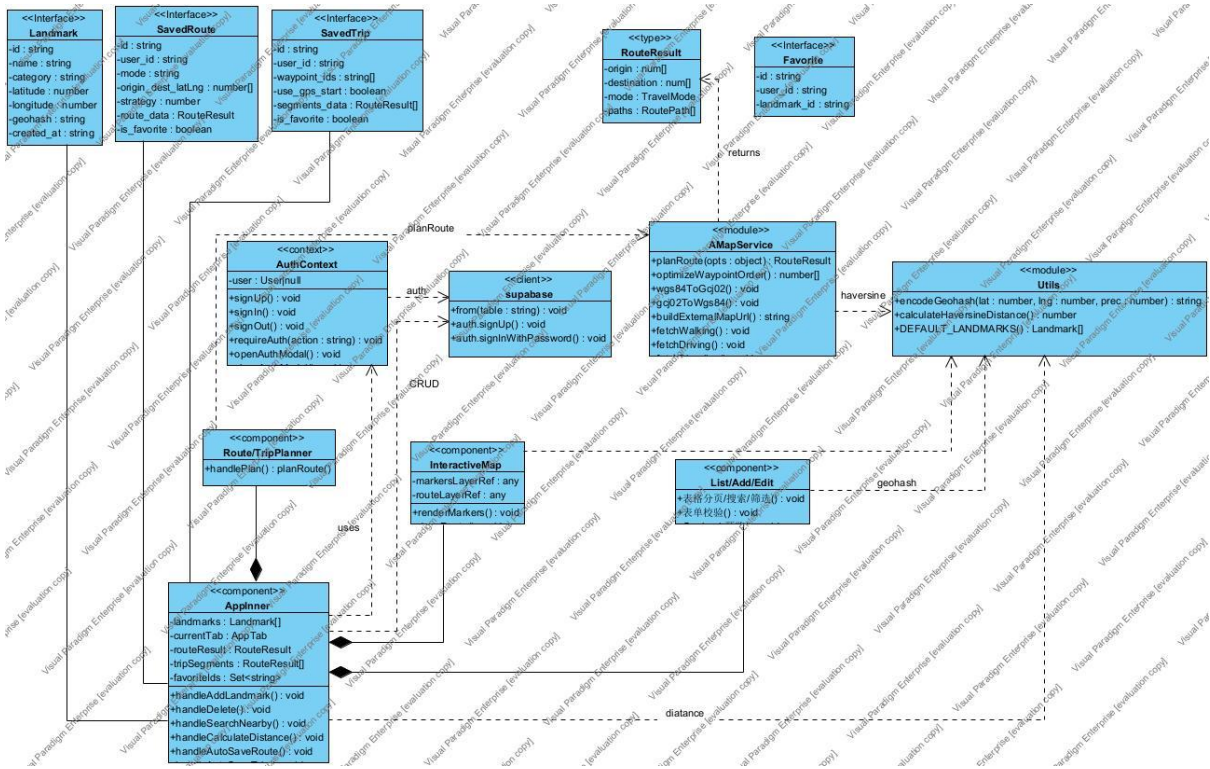


图 4-2 地标管理系统核心类图

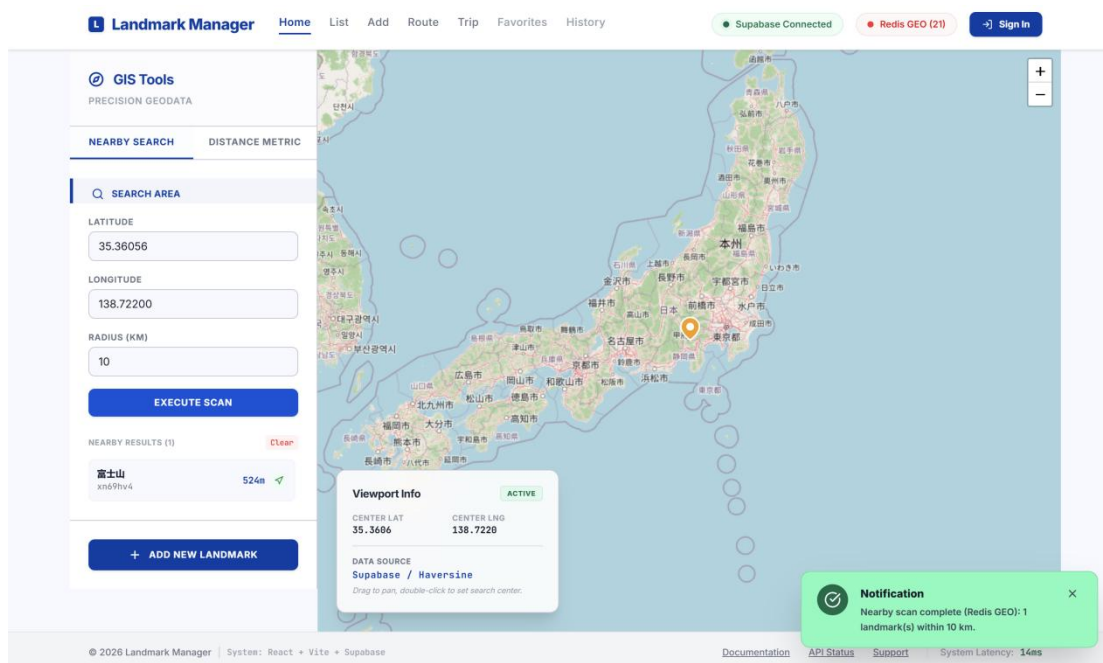
Figure 4-2 Core Class Diagram of the Landmark Manager System

五、系统功能演示与界面截图

本章通过系统实际运行的界面截图，展示各核心功能的交互效果。系统整体采用深色 GIS 主题，以 Inter 与 JetBrains Mono 分别承担界面文字与数据展示，顶部导航栏在 Home、List、Add、Route、Trip、Favorites、History 七个标签间切换，并以状态指示灯实时反映数据库连接状态。

5.1 地图主页与 GIS 工具

地图主页是系统的默认入口，如图 5-1 所示。页面左侧为 GIS 工具侧边栏，集成“附近搜索”与“距离计算”两项功能：用户在附近搜索中输入经纬度与半径并执行扫描，系统基于 Haversine 公式筛选半径内地标并按距离升序列出；在距离计算中选择两个地标，系统计算其球面距离并在地图上以虚线连接、于中点标注距离。右侧地图区域以 OpenStreetMap 为底图，渲染全部地标标记，并在左下角浮动卡片中实时显示当前视口中心坐标。



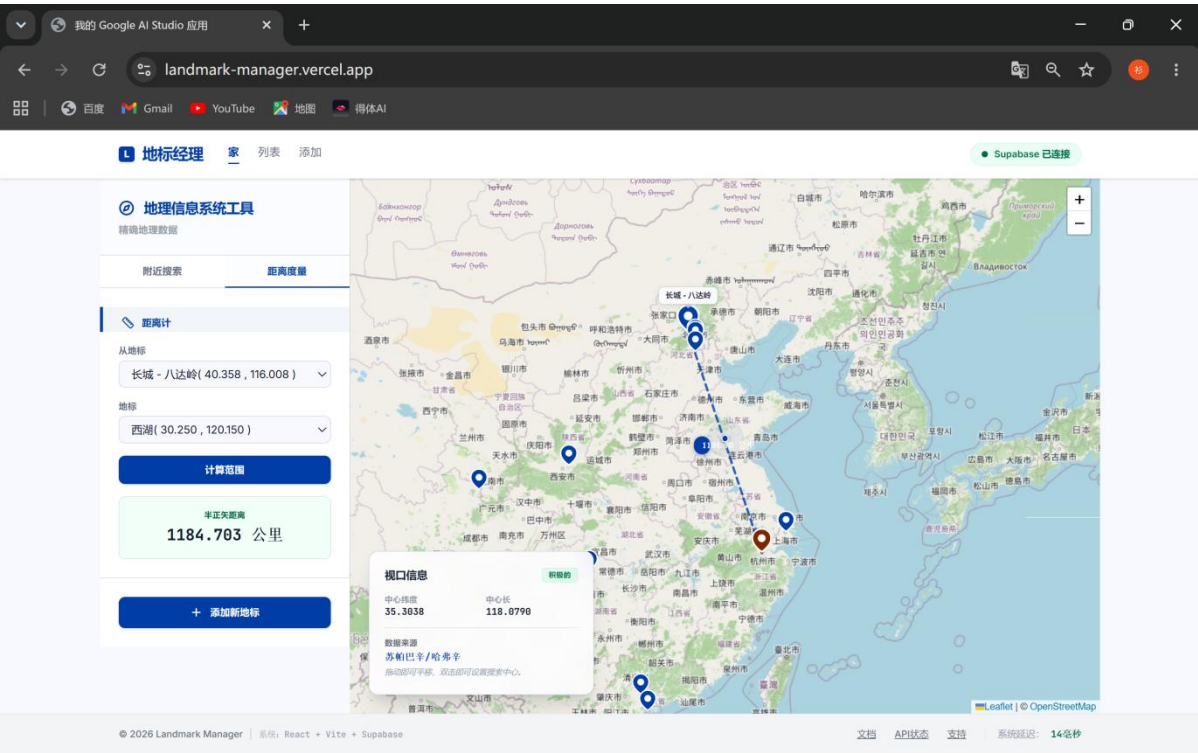


图 5-1 地图主页与 GIS 工具侧边栏

5.2 地标列表管理

地标列表页（图 5-2）以分页表格集中展示全部地标，列出序号、名称、分类、经纬度与 Geohash 等信息。页面顶部提供关键字搜索框与分类筛选下拉，支持对名称、分类与 Geohash 的模糊匹配；表格每行尾部提供收藏、详情、地图定位、编辑与删除等行内操作按钮，删除操作会弹出二次确认以防误操作。列表底部为分页控件，每页展示固定条数记录。

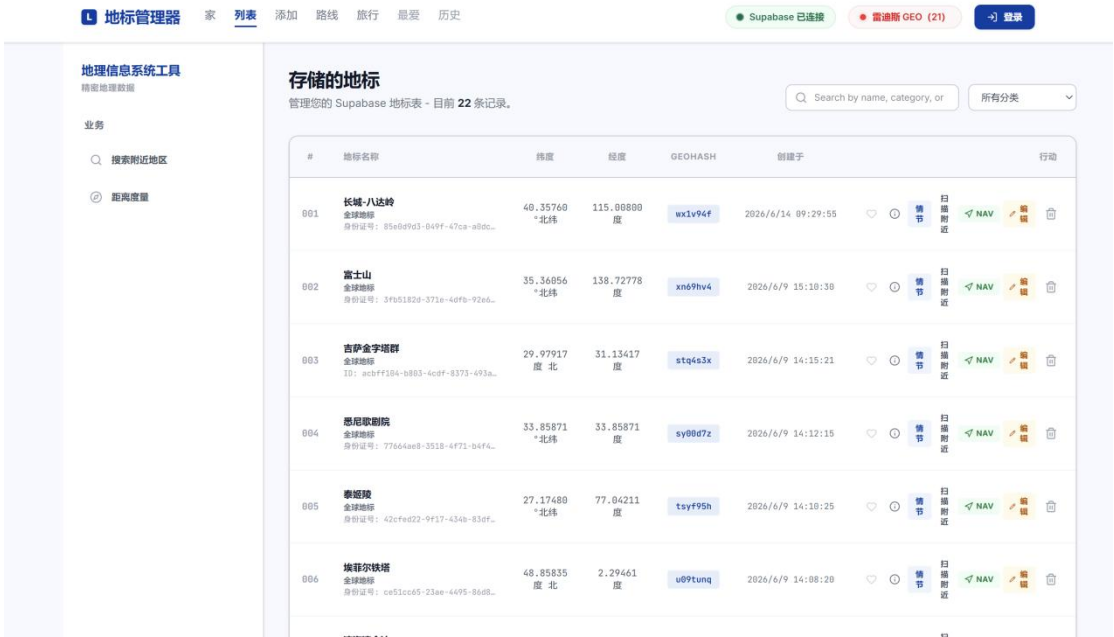


图 5-2 地标列表与搜索筛选

5.3 添加地标与 Geohash 实时预览

添加地标页（图 5-3）提供简洁的表单，用户依次录入名称、分类与经纬度。表单对坐标范围进行实时校验（纬度 $-90\sim90$ 、经度 $-180\sim180$ ），并在下方以醒目的卡片实时显示由当前坐标计算出的 7 位 Geohash 编码——每当经纬度变化，Geohash 即刻重新计算，“LIVE ENCODING”标识直观地反映了这一实时性。提交后系统调用数据库写入并自动跳转至列表页。

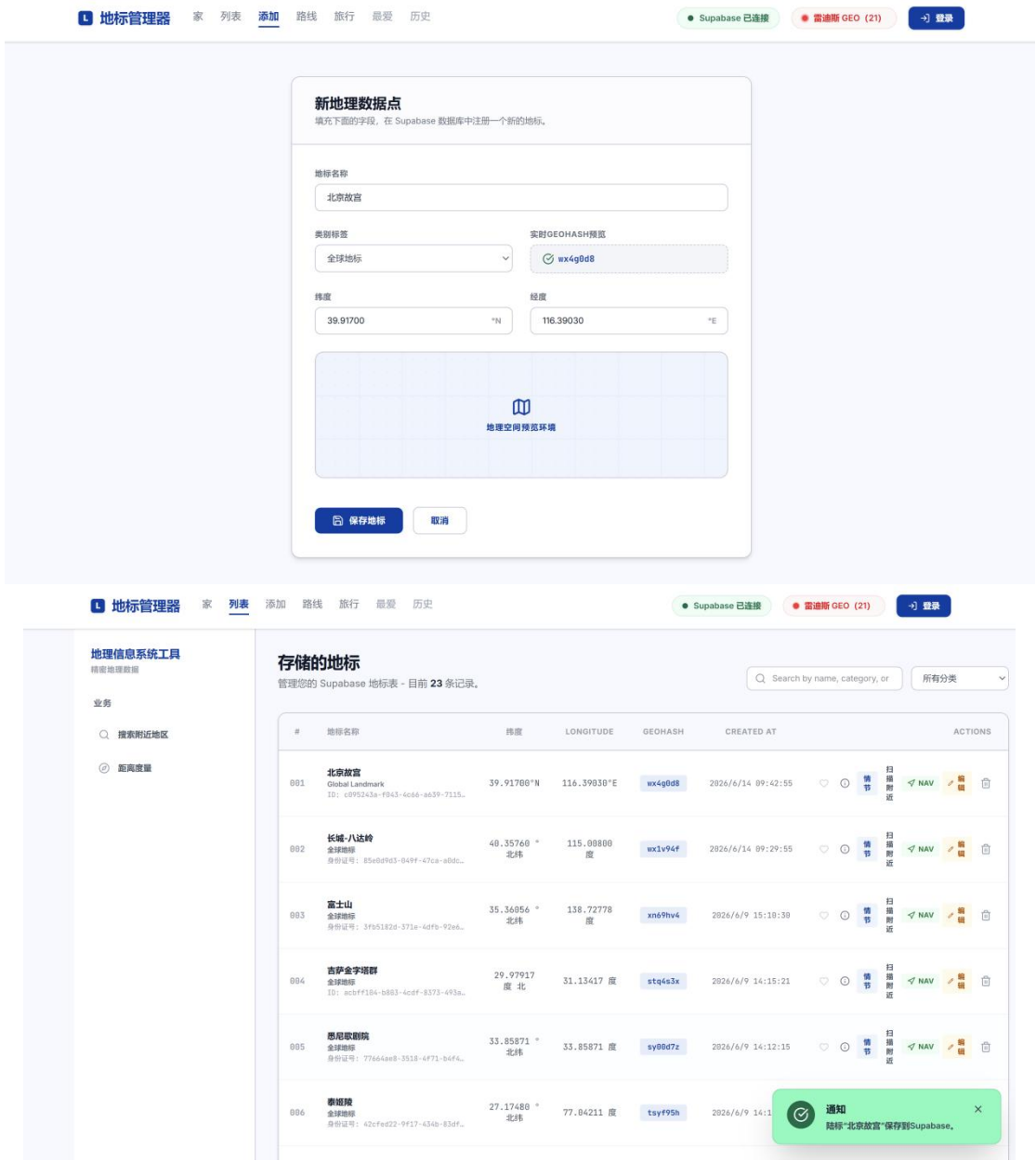


图 5-3 添加地标表单与 Geohash 实时预览

5.4 两点路线规划

路线规划页（图 5-4）支持步行、驾车与骑行三种出行方式，驾车方式下还可进一步选择“速度优先、费用优先、距离优先、躲避拥堵”等策略。起点支持 GPS 定位、从地标选择或手动输入三种方式，终点从地标中选取。点击“Plan Route”后，系统调用高德 API 完成规划，在地图上以蓝色折线绘制路线，并在侧边栏展示总距离、总耗时与逐步导航指令；底部提供高德、百度与苹果地图的外部跳转入口，规划结果同时自动保存至历史记录。

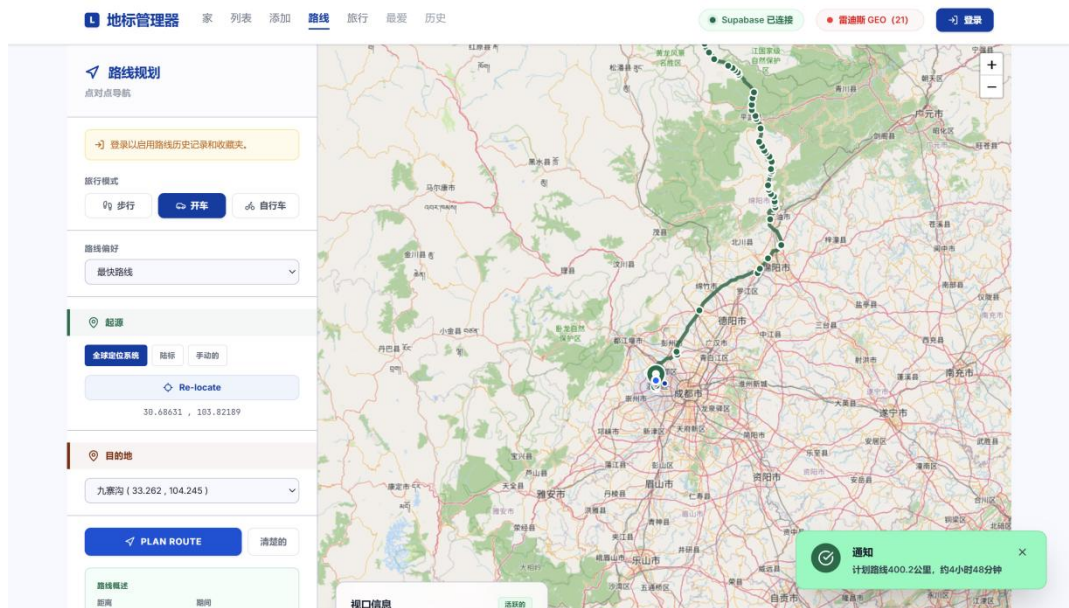


图 5-4 两点路线规划与逐步导航

5.5 多站点行程规划与 TSP 优化

行程规划页（图 5-5）允许用户添加至多 5 个途经点，并通过上移/下移调整顺序或从当前 GPS 位置出发。点击“Optimize”按钮，系统调用最近邻 TSP 启发式自动重排途经顺序，使总行程尽可能短（图中信息卡显示访问顺序由 1→2→3→4 重排为 1→2→4→3）；点击“Plan Trip”后，系统逐段规划相邻点之间的路线，并在地图上以带编号标记与彩色折线呈现完整多段行程，侧边栏汇总总段数、总距离与总耗时。

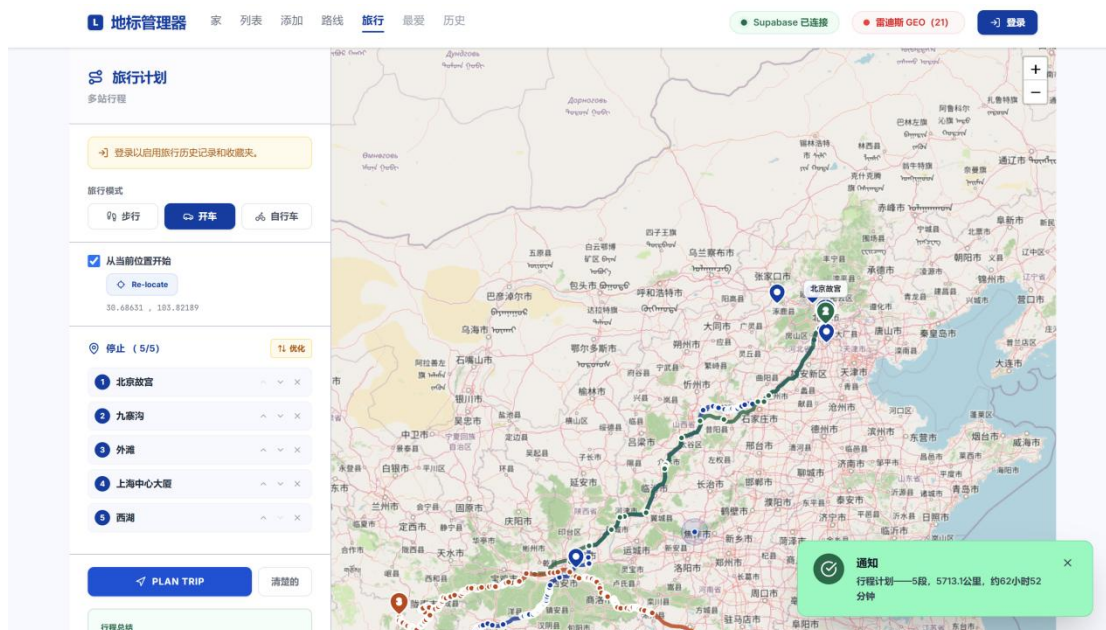


图 5-5 多站点行程规划与 TSP 路径优化

六、系统测试与展望

6.1 功能测试

为验证系统功能的正确性，团队设计了覆盖各模块核心路径的功能测试用例，并以手工黑盒测试的方式逐项执行。部分代表性测试用例及结果如表 6-1 所示，全部用例均通过验证。

表 6-1 代表性功能测试用例

#	测试项	预期结果	结果
1	打开首页加载地图与地标	底图正常渲染，状态灯变绿，标记显示。	通过
2	添加新地标并查看 Geohash	坐标校验有效，Geohash 实时计算，记录入库。	通过
3	编辑地标坐标	Geohash 随坐标实时更新并保存。	通过
4	删除地标	弹出二次确认，确认后记录被删除。	通过
5	附近搜索（经纬度+半径）	返回半径内地标并按距离升序排列。	通过
6	两点距离计算	显示 Haversine 距离并在图上绘制测距线。	通过
7	路线规划（步行/驾车/骑行）	正确绘制路线折线并展示距离、耗时与导航。	通过
8	TSP 行程优化	途经点顺序被自动重排，总距离下降。	通过
9	注册 / 登录	注册提示验证邮件，登录后出现收藏与历史。	通过
10	未登录点击收藏	唤起登录弹窗，登录成功后完成收藏。	通过
11	路线/行程自动保存与回放	历史页出现新记录，可加载回放放到地图。	通过
12	移动端响应式	侧边栏转为底部抽屉，导航折叠为汉堡菜单。	通过

6.2 已知局限

受限于课程项目的时间与定位，本系统仍存在若干不足：其一，高德路线规划 API 仅支持中国境内，境外地标之间无法规划路线（但距离计算不受影响）；其二，TSP 采用贪心近似算法，不保证全局最优，且为控制 API 调用将途经点限制为至多 5

个；其三，地标数据为全局共享，任何访客均可增删改，缺乏更精细的权限控制；其四，免费高德 API 存在每日调用次数限制，GPS 精度亦受设备与环境影响。

6.3 改进展望

后续可从以下方向继续完善：在数据层面，可引入 PostGIS 扩展以支持更高效的空间查询，并为地标增加更丰富的属性与图片；在算法层面，可在 TSP 中引入 2-opt 等局部搜索以改善较多途经点时的解质量；在权限层面，可将地标也纳入用户私有或团队共享的权限体系；在功能层面，可增加地标的批量导入导出、路线的时间窗约束以及多人协作标注等能力，进一步提升系统的实用价值。

七、结论

本文围绕软件工程课程的综合实践目标，设计并实现了一个基于 React 19 + TypeScript + Vite 前端与 Supabase 后端的地标管理与路线规划系统。系统采用前后端分离的 B/S 架构与清晰的分层结构，完整实现了地标的增删改查、基于 Haversine 的附近搜索与测距、Geohash 空间编码、两点路线规划以及含最近邻 TSP 优化的多站点行程规划，并提供了用户认证、收藏与历史记录等个性化功能，共划分为七大功能模块。在工程实践上，五人团队采用 Scrum 与看板相结合的敏捷过程，配合 Git 版本控制与 Vercel 持续部署，完整经历了需求、设计、实现、测试与文档的软件开发全生命周期。

通过本项目，团队不仅产出了一个功能完整、交互流畅、部署便捷的可运行系统，更在真实协作中深化了对软件工程方法论的理解——包括需求的获取与建模、架构与接口的设计权衡、关键算法的工程化实现，以及迭代式过程管理与配置管理的价值。本项目所采用的技术栈与工程方法对同类 Web 全栈应用的开发具有一定的参考意义，而系统在空间查询、路径优化与协作功能上仍有进一步演进的空间。